

Aula 25 - Java 21: Virtual Threads e Pattern Matching

Duração: 4 horas (1h teórica + 3h prática)

Parte Teórica (1 hora)

1. Revisão das Novidades do Java 17 (15 min)

- Records, sealed classes, switch expressions.
- Comparativo com Java 21.

2. Virtual Threads (25 min)

Problema das Threads Tradicionais

- Threads de plataforma (1:1 com threads do SO) são **custosas** (~1MB/thread).
- Limitação prática: ~2000 threads concorrentes.

Solução com Virtual Threads

- Threads leves (green threads) gerenciadas pela JVM (M:N).
- Mesma API (`Thread` e `ExecutorService`).
- **Benefícios:**
 - Suporta milhões de threads concorrentes.
 - Ideal para I/O bound (APIs, bancos de dados).

java

```
// Criação direta
Thread vThread = Thread.startVirtualThread(() -> System.out.println("Hello from virtual t
hread!"));

// Via ExecutorService
try (var executor = Executors.newVirtualThreadPerTaskExecutor()) {
    executor.submit(() -> System.out.println("Task 1"));
    executor.submit(() -> System.out.println("Task 2"));
}
```

3. Pattern Matching (20 min)

Evolução Histórica

- Java 16: Pattern Matching para `instanceof`.
- Java 21: Pattern Matching para `switch` (finalizado).

Exemplos Práticos

java

```
// instanceof simplificado (Java 16+)
if (obj instanceof String s) {
    System.out.println(s.length());
}

// Switch com patterns (Java 21+)
String message = switch (obj) {
    case Integer i -> "Número: " + i;
    case String s && !s.isEmpty() -> "Texto: " + s; // Guarded pattern
    case null -> "Null!";
    default -> "Desconhecido";
};
```

Parte Prática (3 horas)

Atividade: Aplicando Java 21 em um Projeto Spring Boot

Passo 1: Configuração (30 min)

1. Verificar versão do Java:

bash

```
java -version # Deve mostrar >= 21
```

2. Spring Boot 3.2+ (suporta Java 21):

xml

```
<properties>
    <java.version>21</java.version>
</properties>
```

Passo 2: Virtual Threads em API REST (1h30min)

1. Habilitar Virtual Threads (`application.properties`):

properties

```
spring.threads.virtual.enabled=true
```

2. Endpoint concorrente (TaskController.java):

```
java

@GetMapping("/tasks")
public List<Task> getAllTasks() throws InterruptedException {
    try (var executor = Executors.newVirtualThreadPerTaskExecutor()) {
        Future<List<Task>> tasksFuture = executor.submit(() -> taskRepository.findAll
        ());

        Future<Stats> statsFuture = executor.submit(() -> statsService.getStats());

        return tasksFuture.get() // Bloqueia até concluir
            .stream()
            .peek(task -> task.setStats(statsFuture.get()))
            .toList();
    }
}
```

3. Teste de Carga:

- o Use o JMeter ou wrk para comparar com threads tradicionais.

Passo 3: Pattern Matching em Serviços (1h)

1. Processamento de Dados Heterogêneos:

```
java

public String processData(Object input) {
    return switch (input) {
        case User u -> "User: " + u.name();
        case Order o -> "Order value: " + o.value();
        case null, default -> "Invalid data";
    };
}
```

2. Validação com Sealed Classes:

```
java

public sealed interface Shape permits Circle, Rectangle {
    double area();
}

public double calculateTotalArea(List<Shape> shapes) {
    return shapes.stream()
        .mapToDouble(s -> switch (s) {
            case Circle c -> Math.PI * c.radius() * c.radius();
            case Rectangle r -> r.width() * r.height();
        })
}
```

```
        .sum();  
    }
```

Tarefa para Casa

1. Migrar um Service:

- Reescrever um serviço existente usando virtual threads.

2. Refatorar Condicionais:

- Substituir `if/else` por pattern matching em um validator.

Próxima Aula

- **Tópico:** Programação Reativa com Spring WebFlux.
- **Atividade prática:** API não-blocking com R2DBC.

Resultado Esperado

- **Virtual Threads:**
 - Endpoint capaz de lidar com 10x mais requisições concorrentes.
- **Pattern Matching:**
 - Código mais legível e seguro em verificações de tipo.

plaintext

- ✓ Virtual Threads configuradas
- ✓ Pattern Matching em switches
- ✓ Pronto para programação reativa

<https://i.redd.it/java-virtual-threads-vs-platform-threads-v0-5jv8q9vq5wzb1.png> (Comparação de threads) 🚀